



µLab worksheet • Kubernetes in one container

Group _____ Date _____ Time _____

Driver (types) _____ Mapper (looks up) _____ Referee (demos) _____

Hand this in at the end of the round. The referee presents this work in 90 seconds at the caucus, so the referee has to follow it, not just watch it.

1 • THE CORE, IN FOUR STEPS • TEN MINUTES

1 ☐ Start a cluster

```
docker run -d --name k8s --privileged --tmpfs /run --tmpfs /var/run \
-p 6443:6443 rancher/k3s:v1.31.5-k3s1 server --disable=traefik
```

Then `docker exec -it k8s sh` and stay there: `kubectl` is already installed inside.

2 ☐ Ask the cluster to keep a cow running

```
kubectl create deployment cow --image=ghcr.io/<you>/whalesay --replicas=3
```

No image of your own? A neighbour's handle, or `ghcr.io/oesteban/whalesay`.

3 ☐ Watch what it does

`kubectl get pods` — run it twice, about a minute apart. Write both readings in panel 3.

4 ☐ Ask for the same image to run once instead

```
kubectl create job cow --image=<same> -- cowsay 'moo'
```

then `kubectl logs job/cow`.

2 • WHAT YOU STARTED

From `kubectl get nodes`:

NAME _____ VERSION _____

STATUS _____ AGE when first Ready _____

3 • THE TWO READINGS, AND THE REASON

First reading

STATUS _____

RESTARTS _____

A minute later

STATUS _____

RESTARTS _____

The container did not crash. It printed its cow and exited **successfully**. So why is Kubernetes starting it again?

The Job in step 4 used **the same image** and did not do this. What did you change, in one sentence?

Reached this line? Then you have solved the challenge.



Referee: tick the box on the left now, and write the time. Do not wait for the caucus. Then carry on below — the extensions are not optional, they are the rest of the round.

Solved at _____ Referee's initials _____

4 • MAKE THE CLUSTER EXPLAIN ITSELF

First, the node's NAME from panel 2 is not a hostname anybody chose. Where does it come from, and what does that tell you about what a "node" is here?

kubectl describe pod <one of the cow pods> and read the **Events** block at the very bottom. Copy the two most recent event lines.

Then kubectl get events --sort-by=.lastTimestamp | tail -5. Whose account of the morning is this — yours, or the controller's?

5 • TWO IMAGE STORES, ONE NAME

Inside the cluster: crictl images .

On your laptop, in another terminal: docker images .

Cow image present in crictl ? _____ in docker ? _____

Nobody copied anything from your laptop into the cluster. So how did the image get there, and what would have happened if the package were private?

6 • A WISH IS NOT A COMMAND

kubectl scale deployment cow --replicas=5, wait, then kubectl scale deployment cow --replicas=0. Count the pods after each.

pods at 5 _____ pods at 0 _____

Now delete the **Job's** pod: kubectl delete pod <the job pod> . Does it come back? Does a Deployment's pod come back? Why are the answers different?

7 • EXTRA MILE • SATISFY THE DEPLOYMENT INSTEAD OF ABANDONING IT

In step 4 you stopped the restarting by changing the **object**. There is a second way that leaves it a Deployment: same image, no rebuild, no re-push, sitting at `RESTARTS 0` and `Running`. Write the command.

You have now satisfied a Deployment two ways. Which would you use for a web server, and which for a nightly report?

No hint. The image runs one program and then stops; nothing says it has to.

Done when the referee can say, without reading this sheet, why the same image restarts forever as a Deployment and finishes as a Job.